REACT ROUTER

```
const router=createBrowserRouter([
{
  path:"/",
  element:<Home/>
}
,{
  path:"/about",
  element:<About/>
},{
  path:"/dashboard",
  element:<Dashboard/>
}
]);
```

Making a navbar using it

```
const Navbar = () => {
  return (
    <div>
      <ul>
        <li>
          <a href='/'> Home </a>
        </li>
        <li>
          <a href='/about'> About </a>
        </li>
        <li>
          <a href='/dashboard'> Dashboard </a>
        </li>
      </ul>
    </div>
  )
}
```

```
<li>
    <NavLink to='/' className={({isActive})
    => isActive ? "active-link": ""}>
        Home
    </NavLink>
```

in a navbar explain this logic to me

Bhai, is code ka main purpose hai **navbar mein jis page par user currently hai, uske link ko** active-link **class dena**, taaki us link ko alag styling de sakein, jaise orange color, underline, ya bold text.

Chalo ek-ek part samajhte hain.

1. Code ko break karke samjho

```
<NavLink to="/" className={({ isActive }) =>
  isActive ? "active-link" : ""
}>
  Home
</NavLink>
```

Step 1: NavLink to="/"

```
<NavLink to="/">
  Home
</NavLink>
```

- NavLink React Router ka component hai.
- to="/" ka matlab hai Home page ke route par navigate karna.
- Jab user Home par click karega, URL / ho jayega.

Step 2: className={({ isActive }) => ...}

Ye main logic hai.

Normally hum likhte hain:

className="active-link"

Isse class hamesha apply hogi. Lekin navbar mein humein class **sirf tab lagani hai jab user us page par ho.**

React Router NavLink ko ek object deta hai, jisme isActive property hoti hai.

- isActive = true → current URL is link ke route se match karta hai.
- isActive = false → current URL is link ke route se match nahi karta.

Step 3: Ternary operator

isActive ? "active-link" : ""

Iska matlab hai:

condition ? true hone par kya return karein : false hone par kya return karein

Agar isActive **true hai**

"active-link" return hoga.

CSS ki active styling apply hogi.

Agar isActive **false hai**

Empty string "" return hogi.

Active class apply nahi hogi.

2. Real-life example

Maan lo tumhare navbar mein 3 links hain:

<NavLink to="/">Home</NavLink>

<NavLink to="/about">About</NavLink>

<NavLink to="/contact">Contact</NavLink>

Agar current URL /about hai, toh expected result kuch aisa hoga:

Link	isActive	**Class**
Home (/)	false*	""
About (/about)	true	"active-link"
Contact (/contact)	false	""

*React Router mein / route normally other paths ke saath bhi match kar sakta hai; exact Home-only matching ke liye end prop use karo.

3. CSS mein active-link **ka use**

```
.active-link {
  color: orange;
  font-weight: bold;
  border-bottom: 2px solid orange;
}
```

Ab jis page par user hoga, uska navbar link orange aur bold dikhega.

4. Sabse important baat

```
className={({ isActive }) =>
    isActive ? "active-link" : ""
```

}

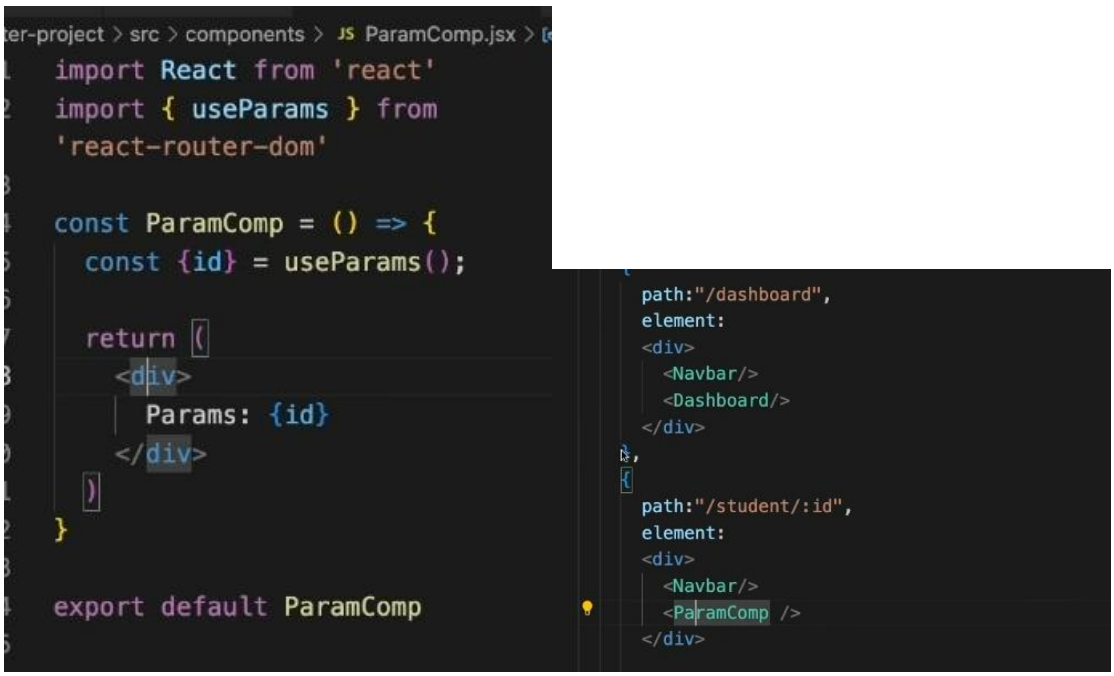
Isko simple English mein yaad rakho:

“NavLink provides an isActive **value based on the current route. If the link is active, we assign the** active-link **CSS class; otherwise, we assign an empty string.”**

Bhai, bas teen cheezein yaad rakhna:

- NavLink → navigation link banata hai.
- isActive → batata hai ki link current route par active hai ya nahi.
- Ternary operator → uske according CSS class choose karta hai.

params



1. Why do we need params?

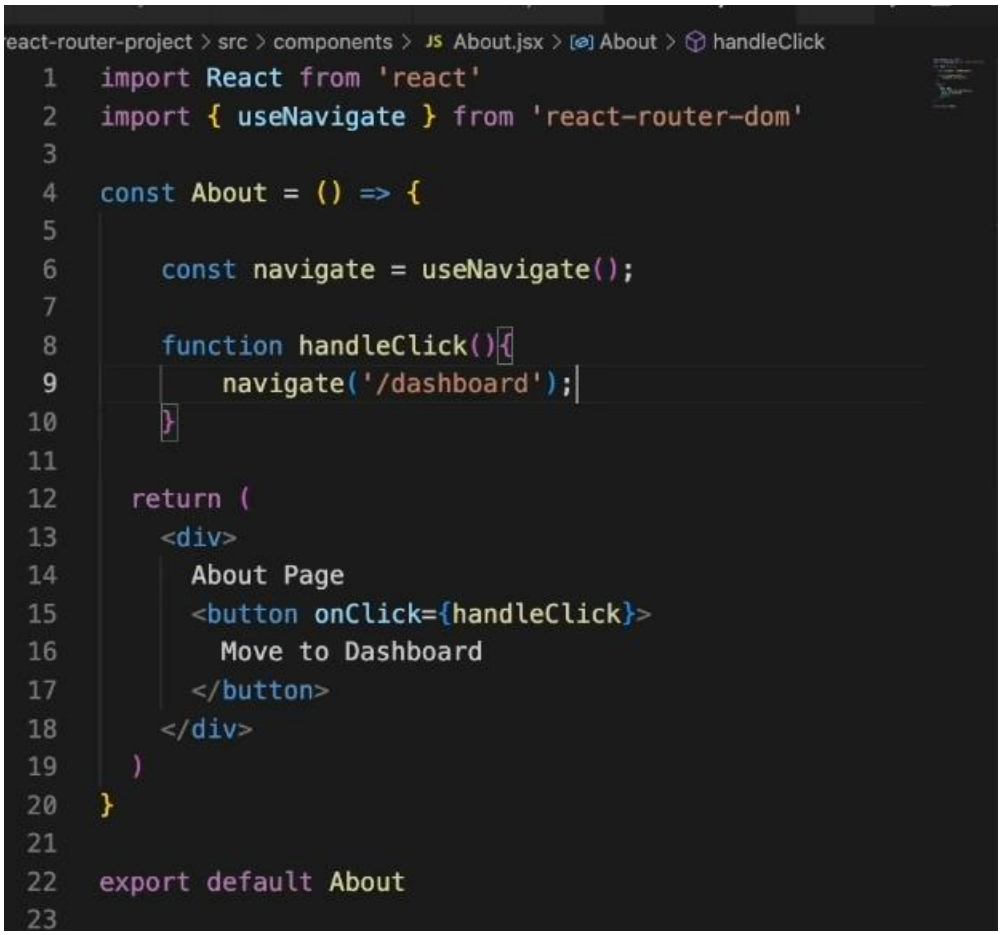
Maan lo tum ek e-commerce website bana rahe ho.

Har product ka apna page hai:

- /product/101
- /product/102
- /product/103

Har product ke liye alag React component banane ki jagah, hum ek hi component use kar sakte hain aur URL se product ki ID nikal sakte hain.

MOVE FROM ONE PAGE TO ANOTHER



Nested routing

```
{
  path:"/dashboard",
  element:
  <div>
    <Navbar/>
    <Dashboard/>
  </div>,
  children:[
    {
      path:'/courses',
      element:
    },
    {
      path:'mock-tests',
      element:
    },
    {
      path:'/reports',
      element:
    }
  ]
```

<Outlet>

▶ Type declaration

An `<Outlet>` should be used in parent route elements to render their child route elements. This allows nested UI to show up when child routes are rendered. If the parent route matched exactly, it will render a child index route or nothing if there is no index route.

```
function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>

      {/* This element will render either <DashboardMessages> when the URL is
          "/messages", <DashboardTasks> at "/tasks", or null if it is "/"
      */}
      <Outlet />
    </div>
  );
}
```